# A Survey of Small Language Models

**Chien Van Nguyen[1]*, Xuan Shen[2*], Ryan Aponte[3*], Yu Xia[4], Samyadeep Basu[5], Zhengmian Hu[5], Jian Chen[6], Mihir Parmar[7], Sasidhar Kunapuli, Joe Barrow[8], Junda Wu[4], Ashish Singh[9], Yu Wang[1], Jiuxiang Gu[8], Franck Dernoncourt[8], Nesreen K. Ahmed[10], Nedim Lipka[8], Ruiyi Zhang[8], Xiang Chen[8], Tong Yu[8], Sungchul Kim[8], Hanieh Deilamsalehy[8], Namyong Park[11], Mike Rimer, Zhehao Zhang[12], Huanrui Yang[13], Ryan A. Rossi[8], Thien Huu Nguyen[1]**

[1]University of Oregon, [2]Northeastern University, [3]Carnegie Mellon University
[4]University of California, San Diego, [5]University of Maryland, College Park
[6]State University of New York at Buffalo, [7]Arizona State University
[8]Adobe Research, [9]University of Massachusetts Amherst, [10]Intel AI Research
[11]Meta AI, [12]Dartmouth College, [13]University of Arizona

arXiv:2410.20011v1 [cs.CL] 25 Oct 2024

## Abstract

Small Language Models (SLMs) have become increasingly important due to their efficiency and performance to perform various language tasks with minimal computational resources, making them ideal for various settings including on-device, mobile, edge devices, among many others. In this article, we present a comprehensive survey on SLMs, focusing on their architectures, training techniques, and model compression techniques.

We propose a novel taxonomy for categorizing the methods used to optimize SLMs, including model compression, pruning, and quantization techniques. We summarize the benchmark datasets that are useful for benchmarking SLMs along with the evaluation metrics commonly used. Additionally, we highlight key open challenges that remain to be addressed.

Our survey aims to serve as a valuable resource for researchers and practitioners interested in developing and deploying small yet efficient language models.

## 1 Introduction

Although large language models (LLMs) have demonstrated impressive performance on a wide array of benchmarks and real-world situations, their success comes at significant cost. LLMs are resource-intensive to train and run, requiring significant compute *and* data. This often means that they are run on centralized and specialized hardware for both training and inference.

As a response to these challenges, there has been a growing interest in small language models (SLMs). Small language models aim to retain the accuracy and/or adaptability of large language models, while being subject to some constraint(s), such as training or inference hardware, data availability, bandwidth, or generation time. Improving model performance relative to these constraints can then improve downstream goals such as privacy, cost, or the ability to run on consumer devices.

The inherent difficulty of a survey of small language models is that the definitions of "small" and "large" are a function of both context and time. GPT-2, a "large language model" in 2019 at 1.5B parameters, is smaller than many "small" language models covered in this survey. However, although the scale changes, the goals of training small language models remain relatively stable.

In this survey, we explore the architectures, training, and model compression techniques that enable the building and inferencing of SLMs. In addition, we summarize the benchmark datasets and evaluation metrics commonly used in evaluating SLM performance. To do this, we propose a novel taxonomy for organizing the methods along two axes:

- the **techniques** used in pre-processing (model architecture), training, and post-processing (model compression) SLMs; and
- the **constraints** the technique is attempting to optimize for, e.g. inference compute, training time, speed, etc.

An overview of these axes can be found in Table 1 (techniques) and Table 2 (constraints).

It is important to note that progress on any one of these goals does not necessarily imply progress on the others. In fact, there are often trade-offs between them. For instance, memory-efficient training methods like quantization-aware training

*The authors contributed equally to this work.

(Dettmers et al., 2022a, 2024) are often slower than their full-precision counterparts. However, by using mixed precision to represent the weights and gradients, they allow training or finetuning using less memory. Finally, although there have been several recent surveys on LLMs and their learning methods (Rogers et al., 2020; Min et al., 2021; Zhu et al., 2023; Shen et al., 2023), to the best of our knowledge, this is the first survey focused on SLMs.

**Organization of the Survey.** This survey is structured into three main sections, each covering a key aspect of optimizing SLMs. **Section 2** focuses on model architectures, including lightweight designs, efficient self-attention approximations, and neural architecture search to efficiently build smaller models. **Section 3** covers efficient pre-training and fine-tuning techniques to enhance performance for SLMs while managing resource constraints. **Section 4** explores model compression techniques, such as pruning, quantization, and knowledge distillation, which reduce model size and latency without sacrificing significant accuracy. **Section 5** introduces an overview of benchmark datasets and evaluation metrics, providing a comprehensive framework for assessing the effectiveness of these methods. **Section 6** discusses the applications that are enabled by SLMs, organized by constraints. Finally, a discussion of open challenges for SMLs is presented in **Section 7**.

**Summary of Main Contributions.** The key contributions of this work are as follows:

- A comprehensive survey of existing work on small language models for practitioners. We also survey the problem settings, evaluation metrics, and datasets used in the literature.
- We introduce a few intuitive taxonomies for SLMs and survey existing work using these taxonomies.
- We identify important applications, open problems, and challenges of SLMs for future work to address.

## 2 Model Architectures

This section discusses the architectural designs for developing SLMs. Specifically, we cover lightweight architectures (Section 2.1),

efficient self-attention approximations (Section 2.2), and neural architecture search (Section 2.3).

### 2.1 Lightweight Architectures

Lightweight language model architectures are designed to achieve efficient performance with fewer parameters and reduced computational overhead, which is ideal for deployment on resource-constrained devices such as mobile phones, edge devices, and embedded systems. Representative lightweight models often follow the encoder-only and decoder-only architectures.

Lightweight encoder-only architectures are mostly optimized versions of BERT (Devlin et al., 2019). For example, MobileBERT (Sun et al., 2020) introduces an inverted-bottleneck structure to maintain a balance between self-attention and feed-forward networks, achieving a 4.3x size reduction and a 5.5x speedup compared to the base version of BERT. DistilBERT (Sanh, 2019) and TinyBERT (Jiao et al., 2019) achieve more than 96

Lightweight decoder-only architectures follow the structure of autoregressive language models such as the GPT (Radford et al., 2018, 2019) and LLaMA series (Touvron et al., 2023b). These models emphasize knowledge distillation, memory overhead optimization, parameter sharing, embedding sharing to enhance efficiency and scalability. BabyLLaMA (Timiryasov and Tastet, 2023a) and BabyLLaMA-2 (Tastet and Timiryasov, 2024) distill knowledge from multiple teachers into a 58M-parameter model and a 345M-parameter model respectively, demonstrating that distillation can exceed teacher models' performance particularly under data-constrained conditions. TinyLLaMA (Zhang et al., 2024), with only 1.1B parameters, achieves high efficiency by optimizing memory overhead, e.g., via FlashAttention (Dao et al., 2022), while maintaining competitive performance for various downstream tasks. MobilLLaMA (Thawakar et al., 2024) applies a parameter-sharing scheme that reduces both pre-training and deployment costs, introducing a 0.5B-parameter model for resource-constrained devices. MobileLLM (Liu et al., 2024e) further introduces embedding-sharing and grouped-query attention mechanisms with block-wise weight sharing to reduce latency.

### 2.2 Efficient Self-Attention Approximations

Deploying large language models can be challenging due to the substantial number of parameters in the self-attention layers, as well as the computational cost associated with self-attention. In this

| Technique | General Mechanism | Training Compute | Dataset Size | Inference Runtime | Memory | Storage Space | Latency |
|---|---|---|---|---|---|---|---|
| **Model Architectures** (Sec. 2) | Lightweight Models (Sec. 2.1) | ✓ | | ✓ | ✓ | | ✓ |
| | Efficient Self-Attention (Sec. 2.2) | ✓ | | ✓ | ✓ | | ✓ |
| | Neural Arch. Search (Sec. 2.3) | | | ✓ | ✓ | ✓ | |
| **Training Techniques** (Sec. 3) | Pre-training (Sec. 3.1) | ✓ | ✓ | ✓ | ✓ | ✓ | |
| | Finetuning (Sec. 3.2) | ✓ | ✓ | | | | |
| **Model Compression** (Sec. 4) | Pruning (Sec. 4.1) | | | ✓ | ✓ | ✓ | ✓ |
| | Quantization (Sec. 4.2) | | | ✓ | ✓ | ✓ | ✓ |
| | Knowledge Distillation (Sec. 4.3) | | ✓ | | | | |

Table 1: General techniques used for optimizing small language models, categorized by type of model optimization and most central constraints they address.

section, we discuss strategies towards decreasing this computational cost which can ultimately be useful in creating small language models.

Reformer (Kitaev et al., 2020) improves the complexity of the self-attention from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ by replacing the dot product attention with one which uses locality-sensitivity hashing. Roy et al. (2021) use a sparse routing module based on an online k-means clustering, which reduces the complexity of the attention computation.

To reduce the computational quadratic complexity of the self-attention layer from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, several works, including (Wang et al., 2020a; Katharopoulos et al., 2020; Xiong et al., 2021; Beltagy et al., 2020), propose linear attention mechanisms. In particular, (Katharopoulos et al., 2020) express self-attention as a linear dot-product of kernel feature maps, thus reducing the quadratic complexity. The authors further show that transformers with this linear attention mechanism can be viewed as a recurrent neural network which enables faster inference. Building on these foundations, recent advancements have led to more advanced architectures. Notable examples include Mamba (Gu and Dao, 2023; Dao and Gu, 2024), which introduces a selective state space model with input-dependent transitions, and RWKV (Peng et al., 2023), which combines elements of transformers and RNNs with a linear attention mechanism. These models not only achieve linear time and space complexity but also demonstrate competitive performance across various tasks. This ongoing trend towards efficient sequence modeling architectures aims to maintain the expressiveness of attention-based models while significantly reducing computational complexity.

We also note some previous work for processing long documents with encoder-only architectures. Longformer (Beltagy et al., 2020) uses a combination of local windowed attention and task-specific global attention which scales linearly with input length, thus being memory efficient. Wang et al. (2020a) approximates the self-attention mechanism using a low-rank matrix which reduces the complexity to $\mathcal{O}(N)$. Both these works show that empirically transformers with linear self-attention matches the performance of the original self-attention mechanism across a variety of downstream tasks. In a similar vein, Xiong et al. (2021) use the popular Nystrom method (Nyström, 1930) for approximating the self-attention operation with strong empirical performances when compared to traditional transformers.

### 2.3 Neural Architecture Search Techniques

This section discusses automated methods to discover the most efficient model architectures for specific tasks and hardware constraints.

Previous research has primarily concentrated on Neural Architecture Search (NAS) for vision tasks (Tan and Le, 2019; Zoph and Le, 2016; Wu et al., 2019; Guo et al., 2020) and BERT models (Xu et al., 2021; Jawahar et al., 2023; Ganesan et al., 2021), as these models have comparatively

fewer parameters, which reduces the cost of the search process for efficient architectures. However, LLMs with over a billion parameters present a significant challenge in searching for smaller, more efficient models. Their massive scale makes the search process computationally intensive and costly. Recently, MobileLLM (Liu et al., 2024e) investigates the impact of model depth (i.e., number of layers) and width (i.e., number of heads) on performance, effectively conducting a targeted architecture search within a smaller parameter range for language models with millions of parameters. Meanwhile, Shen et al. (2024c) reduce the search space by exploring an appropriate initialization for the search, which helps expedite the convergence of the search process.

### 2.4 Small Multi-modal Models

Recent large multi-modal models (LMMs) have achieved comparable or superior performance to their predecessors while significantly reducing the number of parameters. Notable examples include the LLaVA-Next (Liu et al., 2024a), Idefics2 (Laurençon et al., 2024), and InternVL2 (Chen et al., 2023) series. This progress is partly driven by more efficient, smaller language models like Gemma (Team et al., 2024), phi-3-mini (Abdin et al., 2024), and emphasizes the critical role of curated datasets. Additionally, there has been a concerted effort to reduce the size of the vision encoder during multi-modal fusion. InternVL2, for example, leverages outputs from intermediate layers of large visual encoders while discarding the later blocks. Smaller models, such as PaliGemma (Beyer et al., 2024) and Mini-Gemini (Li et al., 2024c), adopt lightweight vision encoders. Monolithic multi-modal models take this further by completely eliminating the visual encoder, instead using lightweight architectures to generate visual tokens. For example, Chameleon (Team, 2024a) employs a VQ-VAE model to encode and decode images into discrete tokens, while Mono-InternVL (Luo et al., 2024a) uses an MLP to generate visual tokens for image patches, incorporating a modality-specific feed-forward network, termed multi-modal Mixture-of-Experts, to differentiate between modalities.

## 3 Training Techniques

This section reviews the key training techniques used for language model pretraining and fine-tuning. While SLMs involve similar training approaches to LLMs, we will focus on efficient techniques to facilitate the general learning scenarios with limited resources for SLMs.

### 3.1 Pre-training Techniques

Mixed precision training is a crucial technique for enhancing pre-training efficiency of SLMs and LLMs. This approach leverages low-precision representations for forward and backward propagation while maintaining high-precision weights for updates. For instance, (Micikevicius et al., 2018) introduced Automatic Mixed Precision (AMP), which initially keeps a master copy of weights in 32-bit floating-point (FP32) precision while performing arithmetic operations in 16-bit floating-point (FP16) precision. However, recent work (Rae et al., 2021) has observed accuracy losses due to its limited numerical range. To address this issue, (Burgess et al., 2019) propose Brain Floating Point (BFLOAT16), offering a greater dynamic range with more exponent bits than FP16. BFLOAT16 has demonstrated superior training performance and representation accuracy compared to FP16. Modern GPU architectures have further advanced mixed-precision capabilities through specialized Tensor Cores. For instance, while earlier generations supported FP16 and BFLOAT16, NVIDIA's latest Hopper architecture introduces support for 8-bit floating-point (FP8) precision (Luo et al.), enabling even greater computational efficiency for large-scale language models.

Complementing these mixed precision approaches, various optimization and stability techniques are employed to prevent model collapse and further enhance training efficiency for SLMs and LLMs. While Adam (Diederik, 2014) and AdamW (Loshchilov and Hutter, 2019) optimizers are commonly used, memory-efficient variants like Adafactor (Shazeer and Stern, 2018) and Sophia (Liu et al., 2024b) have been introduced to improve training speed and efficiency. To further stabilize training, gradient clipping (Zhang et al., 2020) is widely used to prevent exploding gradients. Additionally, careful initialization strategies can provide a good starting point for model training. These combined techniques aim to achieve optimal training efficiency, maintain numerical stability, and produce more robust and capable language models.

To address the computational demands of the pre-training stage, language models are typically pre-trained across multiple machine nodes, lever-

aging distributed computing resources efficiently. Several system-level optimization techniques have been developed to this end. Zero Redundancy Data Parallelism (ZeRO) (Rajbhandari et al., 2020) offers three progressive stages of optimization, each partitioning more training states across devices: ZeRO-1 partitions optimizer states, ZeRO-2 adds gradient partitioning, and ZeRO-3 further partitions model parameters. PyTorch's Fully Sharded Data Parallel (FSDP) (Zhao et al., 2023b) implements similar concepts. These parallelism techniques enable training with larger batch sizes, significantly improving efficiency and scalability for SLMs and LLMs.

### 3.2 Fine-tuning Techniques

Fine-tuning on smaller, task-specific datasets allows LLMs to leverage the knowledge gained during pre-training, enabling them to excel in specialized tasks or domains. Fine-tuning techniques are designed to address challenges like limited computing resources, data quality, availability, and robustness, ensuring efficient adaptation to new tasks without extensive retraining.

#### 3.2.1 Parameter-Efficient Fine-Tuning

Parameter-Efficient Fine-Tuning (PEFT) updates a small subset of parameters or adds lightweight modules, keeping most of the pre-trained model's parameters fixed. This approach reduces computational costs during SLM fine-tuning, preserves the model's knowledge, reduces overfitting, and improves flexibility. LoRA uses low-rank decomposition (Hu et al., 2021), Prompt Tuning (Lester et al., 2021) inserts learnable prompts into inputs, and Llama-Adapter (Zhang et al., 2023b; Gao et al., 2023) adds prompts to LLaMA's attention blocks. Dynamic Adapters (Kong et al., 2024; Feng et al., 2024; Gou et al., 2023; Liu et al., 2023b; Luo et al., 2024b) automatically combine multiple adapters as a mixture-of-experts model to enable multi-tasking and prevent forgetting (Han et al., 2024; Yang et al., 2024).

#### 3.2.2 Data Augmentation

Data augmentation increases the complexity, diversity and quality of training data, leading to improved generalization and performance on downstream tasks. AugGPT (Dai et al., 2023) rephrases training samples using ChatGPT. Evol-Instruct (Xu et al., 2023) uses multistep revisions to generate diverse, open-domain instructions with increased complexity. Reflection-tuning (Li et al., 2023a, 2024a) enhances data quality and instruction-response consistency for instruction tuning by refining both instructions and responses using GPT-4 based on predefined criteria. FANNO (Zhu et al., 2024) augments instructions and generates responses by incorporating external knowledge sources through retrieval-augmented generation. LLM2LLM (Lee et al., 2024b) generates more hard samples based on model prediction on training data during training.

Data augmentation is also effective for synthesizing new data when training data is limited, such as for low-resource languages (Whitehouse et al., 2023), medical and clinical applications (Chintagunta et al., 2021), and privacy-sensitive data (Song et al., 2024), enabling models to generalize better and perform more robustly in constrained settings.

## 4 Model Compression Techniques

Model compression techniques focus on reducing the size and complexity of large pre-trained language models while maintaining their performance. As a result, these methods are a key approach to deriving SLMs from LLMs. In this section, we propose a taxonomy for model compression that categorizes such techniques by whether they perform pruning (Section 4.1), quantization (Section 4.2), or knowledge distillation (Section 4.3).

### 4.1 Pruning Techniques

Weight pruning is a model optimization technique that reduces the number of parameters to enhance computational efficiency and lower memory usage, all while maintaining performance levels. We differentiate between two major approaches for pruning: unstructured pruning and structured pruning.

**Unstructured pruning** removes less significant individual weights, offering fine-grained control and flexibility in reducing model size. For example, to perform irregular pruning on large language models, SparseGPT (Frantar and Alistarh, 2023) reformulates the pruning task as a sparse regression problem, optimizing both the remaining and pruned weights using a layer-wise approximate regression solver. SparseGPT can efficiently handle large-scale models like OPT-175B and BLOOM-176B. Additionally, (Boža, 2024) integrates the ADMM (Boyd et al., 2011) algorithm for weight updates to further mitigate pruning errors. Wanda (Sun et al., 2023) incorporates both

weights and activations into consideration during pruning process, and eliminates the need of weight updates. The n:m pruning strategy (Zhou et al., 2021) brings unstructured pruning to model acceleration by pruning exactly $n$ weights out of every $m$, balancing pruning flexibility and computational efficiency for significant speedups. NVIDIA's TensorRT leverages such sparse patterns to optimize memory access and reduce computational loads, accelerating inference on GPUs, particularly hardware like the A100. Notably, unstructured pruning often results in sparse matrices requiring specialized hardware or algorithms to maximize computational benefits (Frantar and Alistarh, 2023).

**Structured pruning** (Wang et al., 2020b; Santacroce et al., 2023; Ma et al., 2023; Tao et al., 2023; Xia et al., 2024; Kurtić et al., 2024) aims to compress LLMs while maintaining performance by removing groups of parameters in a structured manner, which enables more efficient hardware implementation. A major direction in this approach concerns the sparsity of neurons in the model. For instance, Li et al. (2023b) observes prevalent sparsity in feed-forward networks. Liu et al. (2023e) proposes using small neural networks for dynamic pruning based on input, termed "contextual sparsity". Mirzadeh et al. (2024) change the activation functions in pre-trained models to ReLU and fine-tune to improve activation sparsity.

Recent work has also addressed the redundancy in the Transformer architecture to achieve reduction of GPU memory usage and speed enhancement (Michel et al., 2019; Voita et al., 2019; Ge et al., 2024). For example, Sajjad et al. (2023); Xia et al. (2022) investigates the layer redundancy for effective structured pruning. We also highlight input-dependent pruning methods, such as contextual sparsity (Liu et al., 2023e) and FastGen (Ge et al., 2024), which should be considered along with the challenges of efficient implementation for optimizing computation and memory. Appendix A provides further discussion of pruning techniques.

### 4.2 Quantization

Quantization is widely adopted to compress LLMs with vast parameter counts. The GPTQ (Frantar et al., 2022) focuses on layer-wise weight-only quantization, using inverse Hessian matrices to minimize the reconstruction error. To fully leverage the benefits of fast integer matrix multiplication, more quantization methods (Liu et al., 2023a; Dettmers et al., 2022b; Kim et al., 2023; Xiao et al., 2023; Yao et al., 2022; Lin et al., 2024; Liu et al., 2023d, 2024d, 2023c; Shao et al., 2023) that quantize both weights and activations are increasingly being adopted for LLMs. AWQ (Lin et al., 2024) and ZeroQuant (Yao et al., 2022) take activation into account to assess the importance of weights, enabling more effective optimization for weight quantization. In addition, for K/V Cache Quantization (Hooper et al., 2024; Liu et al., 2024f; Yue et al., 2024), Key-Value cache is specifically quantized for enabling efficient long-sequence length inference.

Another challenge of activation quantization lies in the outliers that fall outside the typical activation distribution. SmoothQuant (Xiao et al., 2023) smoothes activation outliers by migrating quantization difficulty from activations to weights. SpinQuant (Liu et al., 2024d) introduces rotation matrices to transform outliers into a new space. Recently, quantization-aware training (QAT) methods, such as LLM-QAT (Liu et al., 2023d) and EdgeQAT (Shen et al., 2024b), have gained attention due to the strong performance. Both methods adopt distillation with float16 models to recover the quantizationi error. We also note recent work (Shen et al., 2024a,b; Zeng et al., 2024) that implements the quantized LLMs on mobile devices and FPGAs to demonstrate the effectiveness and efficiency of the weight and activation quantization for LLMs.

### 4.3 Knowledge Distillation Techniques

In its classical form, knowledge distillation (Hinton et al., 2015) involves training an efficient model, known as the "student," to replicate the behavior of a larger, more complex model, referred to as the "teacher." In this section, we particularly focus on distillation strategies from one or multiple white-box teacher language model to a target student language model.

Babyllama (Timiryasov and Tastet, 2023b) is among the first to develop a compact 58M parameter language model using a Llama model as the teacher. A key finding of this work is that distillation from a robust teacher can outperform traditional pre-training on the same dataset. In a similar vein, (Gu et al., 2024) introduce modifications in the distillation loss, which enables the student models to generate better quality responses with improved calibration and lower exposure bias. Sequence-level distillation loss can

also be improved by using a generalized version of f-divergences as shown in (Wen et al., 2023). Liang et al. (2023) extend layer-wise distillation strategies for language models by using task-aware filters which distill only the task specific knowledge from the teacher. Recent works (Wan et al., 2024a,b) show that multiple language models can be fused as a teacher towards distilling knowledge into small language models by strategically merging their output probability distributions.

One of the issues in knowledge distillation for language models is that the distillation strategies are primarily effective when (1) the teacher and the student language model share the same tokenizer and (2) the teacher's pre-training data is available. Boizard et al. (2024) addresses this issue by introducing an universal logit distillation loss inspired from the optimal transport literature. Often distillation is also combined with pruning techniques towards creating smaller language models. For example, (Sreenivas et al., 2024; Muralidharan et al., 2024) show that an iterative step of pruning a large language model followed by retraining with distillation losses, can enable strong smaller models.

Recent advancements have explored methods beyond traditional label distillation by incorporating additional supervision during the distillation process to create smaller language models. Hsieh et al. (2023) find that using "rationales" as an additional source of supervision during distillation makes it more sample-efficient. Moreover, the authors find that the distilled model outperforms large-language models on commonly used NLI, Commonsense QA and arithmetic reasoning benchmarks. In a similar vein, (Dai et al., 2024; Magister et al., 2023; Ho et al., 2023; Fu et al., 2023) distill the reasoning chain from a larger language model to a smaller language model along with the label information. Such distilled models have been shown to possess improved arithmetic, multi-step math, symbolic and commonsense reasoning abilities.

## 5 Evaluation

Table 2 presents different evaluation settings along with their corresponding datasets and metrics for SLMs. In this section, we examine how different datasets and evaluation metrics are specifically designed to assess SLMs. These evaluation components are organized according to the constraints they address for SLMs.

### 5.1 Datasets

The datasets commonly used for pre-training and evaluating SLMs across various settings are outlined in Table 2. These datasets provide diverse contextual examples that enable models to generalize effectively across different learning settings.

**Efficient Inference** This setting requires models to generate output as quickly as possible, with minimal latency and high throughput. Evaluation datasets for this setting often focus on tasks that require fast response times, such as question answering, text classification, and natural language understanding. To this end, some of the example evaluation datasets for this setting can include SuperGLUE (Sarlin et al., 2020), SQuAD (Rajpurkar et al., 2016), TriviaQA (Joshi et al., 2017), CoQA (Reddy et al., 2019), Natural Questions (NQ) (Kwiatkowski et al., 2019), and many more (Chang et al., 2024) that cover various tasks that require faster response time.

**Privacy-preserving** Privacy-preserving datasets play an important role in enabling the development of SLMs while safeguarding sensitive information. Datasets such as PrivacyGLUE (Shankar et al., 2023) apply differential privacy techniques to common tasks such as sentiment analysis. Anonymized datasets such as MIMIC (Johnson et al., 2020) and n2c2 datasets[1] contain de-identified clinical notes for medical tasks, protecting personal health information. Additionally, federated datasets such as LEAF[2] allow data to remain distributed across devices, supporting privacy by design through federated learning frameworks.

**TinyML and On-device** In these settings, the focus is on deploying SLMs in highly resource-constrained environments. Frameworks such as TinyBERT (Jiao et al., 2020) and OpenOrca (Lian et al., 2023) play a pivotal role by enabling the training and evaluation of SLMs on curated datasets tailored for such environments. TinyBERT, a distilled version of BERT, is optimized for both size and speed, making it suitable for on-device applications with minimal latency requirements. Similarly, subsets like OpenOrca provide useful datasets that balance performance and resource constraints, supporting the development of small, efficient models

[1] https://portal.dbmi.hms.harvard.edu/projects/n2c2-nlp/

[2] https://github.com/TalwalkarLab/leaf

| Setting | Constraints | Datasets | Metrics |
|---|---|---|---|
| Efficient Inference | Latency | SuperGLUE (Sarlin et al., 2020), SQuAD (Rajpurkar et al., 2016), TriviaQA (Joshi et al., 2017), CoQA (Reddy et al., 2019), Natural Questions (NQ) (Kwiatkowski et al., 2019) | Inference Time (Narayanan et al., 2023), Throughput (Arora et al., 2024) |
| On-device/Mobile | Memory | TinyBERT (Jiao et al., 2020) and OpenOrca (Lian et al., 2023) | Peak Memory Usage (Lee et al., 2024a), Memory Footprint, Compression Ratio (Cao et al., 2024) |
| Privacy-Preserving | Privacy | PrivacyGLUE (Shankar et al., 2023), MIMIC (Johnson et al., 2020) | Privacy Budget (Yu et al., 2024), Noise Level (Havrilla et al., 2024) |
| Energy-Efficient AI | Energy Optimization | - | Energy Efficiency Ratio (Stojkovic et al., 2024b), Thermal Efficiency, Idle Power Consumption (Patel et al., 2024) |

Table 2: Overview of Settings, Constraints, and Metrics.

that can be deployed on low-power devices without sacrificing accuracy.

### 5.2 Metrics

The key metrics for evaluating SLMs across different settings are presented in Table 2. The evaluation metrics are organized based on the specific constraints.

**Latency** Two key metrics to evaluate latency are inference time (Narayanan et al., 2023) and throughput (Arora et al., 2024). Inference time measures how quickly a model can process input and generate an output, which is crucial for user-facing applications that require immediate feedback. Throughput, on the other hand, evaluates the number of tokens or samples a model can process in a given period, making it especially relevant for large-scale tasks or time-sensitive applications.

**Memory** When deploying models in memory-constrained environments, memory efficiency becomes a primary consideration. Metrics such as peak memory usage (Lee et al., 2024a) capture the highest amount of memory the model consumes during inference. Similarly, memory footprint and compression ratio (Cao et al., 2024) are used to measure how compact a model is and the efficiency of the compression techniques applied, enabling models to operate within memory constraints without sacrificing performance.

**Privacy** Privacy budget (Yu et al., 2024), a measure rooted in differential privacy, quantifies the model's ability to protect sensitive information during both training and inference. Alongside this, noise level (Havrilla et al., 2024) measures the trade-off between privacy and accuracy by assessing how much noise is added to ensure privacy while maintaining the model's performance.

**Energy Optimization** The energy efficiency ratio (Stojkovic et al., 2024b) evaluates the energy used relative to the model's overall performance, providing insights into how energy-intensive an SLM is in practice. Other metrics, such as thermal efficiency and idle power consumption (Patel et al., 2024), measure the energy consumed when the model is either actively processing tasks or idle, which is crucial for long-term deployment in energy-constrained environments like embedded systems or mobile devices.

## 6 Applications

In this section, we consider applications of SLMs, that is, specific use-cases like translation and auto-completion.

### 6.1 Real-Time Interaction

GPT-4o, released in May 2024, processes text, vision, and audio input end-to-end and is faster than GPT-4 Turbo (OpenAI, 2024b). The demonstration involved responses in the style of human conversation. LLaMA-Omni combine a speech encoder, adaptor, LLM, and streaming decoder to enable real-time interaction with speech input based on LLaMA-3-8B-Instruct (Fang et al., 2024). Emotionally Omni-present Voice Assistant, or EMOVA, apply LLaMA-3.1-8B as an end-to-end speech model that can generate poems and describe images at the user's request. Google Deepmind's Project Astra uses Gemini to process audio and video information from a smartphone or glasses and respond to respond to queries like mathematics problems and memorize object sequences (Deepmind, 2024).

### 6.2 Content Generation and Processing

LLMR uses LLMs in mixed reality to generate and modify 3D scenes. It combines language models used in several roles - a Scene Analyzer GPT

| Category | Application | Need for SLM Application | Inference Runtime | Memory | Storage Space | Latency | Comm. Overhead |
|---|---|---|---|---|---|---|---|
| Real-Time Interaction | **Chatbots** | Real-time response needed, lightweight | ✓ | ✓ | | ✓ | ✓ |
| | **Voice Interfaces** | Low latency required for real-time | ✓ | ✓ | | ✓ | |
| | **Translation** | Real-time translation with low-resources | ✓ | ✓ | | ✓ | ✓ |
| Content Generation & Processing | **Text Summarization** | Faster inference, minimal resource use | ✓ | ✓ | ✓ | ✓ | |
| | **Sentiment Analysis** | Efficient analysis in low-resource envir. | ✓ | ✓ | ✓ | ✓ | |
| | **Text Classification** | Low latency, on-the-fly processing | ✓ | ✓ | ✓ | ✓ | |
| | **NLP for Search** | Low latency for real-time search | ✓ | ✓ | | ✓ | |
| | **Autocompletion** | Fast prediction with low memory | ✓ | ✓ | ✓ | ✓ | |

Table 3: Taxonomy of Applications of Small Language Models.

to summarize objects and give further details like color, Skill Library GPT to determine what is required to fufill a user's request, Builder GPT to generate code for the request, and Inspector GPT to evaluate its code (Torre et al., 2024). DreamCodeVR assists users in editing an application in the Unity engine through code generation (Giunchi et al., 2024; Juliani et al., 2020). This permits users to edit VR applications without requiring extensive programming knowledge.

### 6.3 Edge Inference and Privacy

On-device LLMs maintain usability even when MobileLLM improve on various chat benchmarks and performs comparably with LLaMA-2-7B in API calling (Liu et al., 2024e). Apple Intelligence applies an 3B parameter model to perform on-device inference for a broad range of tasks, such as text and notification summarization, image and emoji generation, and code completion for XCode (Gunter et al., 2024; Research, 2024). On-device inference reduces latency as measured by the time to first generated token (Hu et al., 2024; Gerganov). HuatuoGPT is a domain-adapted LLM for medical dialogue and BioMistral is an LLM tailored for biomedical work (Zhang et al., 2023a; Labrak et al., 2024). Applications related to medicine may need to adhere to stringent privacy regulations and represent a promising area for future work. TalkBack with GeminiNano assists blind and low vision people by describing and captioning images and runs on Android devices (Team, 2024b). On-device inference makes this technology usable without an internet connection.

Mixture-of-Experts can reduce inference cost by using a gating network to use only a subset of layers during inference time (Shazeer et al., 2017). Google's GLaM uses mixture-of-experts (Du et al., 2022) but is a 1.2T parameter model. EdgeMoE extend misture-of-experts to edge computing using an Nvidia Jetson TX2 and Raspberry Pi 4B, with the latter device being CPU-only (Sarkar et al., 2023). Based on experimental findings that most weights contribute little to the final computation, the authors compress weights and predict the relevant experts in advance.

## 7 Open Problems

In this section, we discuss open problems and highlight important areas for future work. Hallucination and bias are a concern shared by SLMs and LLMs (Section 7.1 and 7.2). In Section 7.3, we discuss the increased demand of energy efficiency during inference. Finally, we examine the privacy risks of SLMs in Section 7.4.

### 7.1 Hallucination

A pervasive problem with LLMs is hallucination, defined as content that is nonsensical or untruthful in relation to certain sources (OpenAI, 2024a). OpenAI (2024a) propose that as users rely more on models, the harm caused by hallucinations may be increased. Hallucination can be classified into two types: factuality and faithfulness (relevance). With hallucination of factuality, the generation is inconsistent with verifiable facts. In faithfulness

hallucination, generation lacks relevance to user queries (Huang et al., 2023). HallusionBench, a benchmark for image-context reasoning in vision-language models, found that larger sizes reduced hallucinations (Guan et al., 2024). Analysis of the AMBER hallucination benchmark find that the type of hallucination varies as parameter count changes in Minigpt-4 (Wang et al., 2024). However, find that bias increases with parameter count for the LLaMA series of models (Zhao et al., 2023a). Future work may need to consider not only how total hallucinations change in SLMs, but also the type and severity may be influenced by model size.

### 7.2 Biases

Language models have been found to reproduce biases present in training data (Brown et al., 2020; OpenAI, 2024a; Touvron et al., 2023a).

**Measuring Bias** Methods for measuring bias such as Bias Benchmark for Question Answering (BBQ) (Parrish et al., 2022), RealToxicityPrompts (Gehman et al., 2020), and Crowd-sourced Stereotype Pairs benchmark (CrowS-Pairs) (Nangia et al., 2020).

**Influence of Parameter Count** (Touvron et al., 2023a) find that larger LLaMA models exhibit increased measured bias on RealToxicityPrompts. (Zhao et al., 2023a) replicate this with StereoSet (Nadeem et al., 2021) and their metric GPT-BIAS, which uses GPT-4 to classify responses as biased or unbiased. For comparable model sizes, LLaMA-2 had less measured bias than the previous generation (Touvron et al., 2023c).

### 7.3 Inference-time Energy Use

Energy efficiency is a high priority for SLMs, especially when used on battery-powered devices. Husom et al. (2024) find that architecture significantly influences power consumption using the MELODI benchmar. CPU-only inference was found to be generally less efficient than on GPU and that laptops require more energy for inference. The authors find response token length to be the most effective predictor of energy usage, suggesting that more concise responses can help to extend battery life. Stojkovic et al. (2024a) find that energy usage can be reduced by about 20

### 7.4 Data Privacy

Privacy concerns can be broadly classified into three categories: training data, the system prompt used at inference time, and the user query. Query privacy is especially important in SLMs.

**Training Data** Li et al. (2024b) address training and system prompt leaking. The authors find that the risk of training data leakage increased faster than their measure of utility for the model series Pythia (Biderman et al., 2023). They also find that data towards the end of pre-training is easier to extract, with attention layers as a possible cause.

**System Prompt** Liu et al. (2024c) describe unauthorized retrieval of the system prompt as prompt leaking and use of the prompt for unintended purposes as prompt abuse. They give the example of getting a prompt designed to rephrase user queries to generate code, leading to unexpected cost using Pear AI[3].

**Inference-time Data** Unlike with the leakage of training data and the system prompt, this primarily impacts the end-users of a model. In June 2024, Apple announced the application of language models to the digital assistant Siri (Research, 2024). In the context of digital assistants, SLMs may need to interface with user data like location history or protected health information. If such data were used to train or protect a model from misuse, users might face externalities. Existing literature is limited.

## 8 Conclusion

Given the growing importance of SLMs due to their efficiency and applicability across a wide range of devices and environments, this paper has surveyed SLMs including model architectures, training techniques, and model compression techniques for optimizing SLMs. We also introduced an intuitive taxonomy of evaluation metrics for SLMs and summarize various settings and applications where they are important. Furthermore, we summarized the training and benchmark datasets that have been used for SLMs. Finally, we highlighted the fundamental challenges and open problems that remain to be addressed. We hope this survey serves as a valuable resource for both researchers and practitioners. driving the next advancements in small yet powerful language models.

## 9 Limitations

While SLMs present a broad array of benefits, risks and limitations must also be considered. Hallucina-

[3] https://www.parea.ai

tion (discussed in Section 7.1) and reinforcement of societal biases (discussed in Section 7.2) are widely recognized risks of large language models. While research has been performed to measure and reduce these behaviors, they have yet to be fully mitigated. Utama et al. (2020) introduce a framework to reduce self-bias without the specific bias known at test time. Such methods may become more effective with general increases in model capability. However, risks specific to groups from which researchers are not primarily drawn may remain unrecognized.

## References


Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Sinan Kaplan, Megan Leszczynski, Isys Johnson, Vishal Subbiah, Azalia Mirhoseini, James Zou, and Christopher Ré. 2024. Simple linear attention language models balance the recall-throughput tradeoff.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *CoRR*, abs/2004.05150.

Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. 2024. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR.

Nicolas Boizard, Kevin El Haddad, Céline Hudelot, and Pierre Colombo. 2024. Towards cross-tokenizer distillation: the universal logit distillation loss for llms.

Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. 2011. [link].

Vladimír Boža. 2024. Fast and optimal weight update for pruned large language models. *arXiv preprint arXiv:2401.02938*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners.

Neil Burgess, Jelena Milanovic, Nigel Stephens, Konstantinos Monachopoulos, and David Mansell. 2019. Bfloat16 processing for neural networks. In *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*, pages 88–91. IEEE.

Zhiwei Cao, Qian Cao, Yu Lu, Ningxin Peng, Luyang Huang, Shanbo Cheng, and Jinsong Su. 2024. Retaining key information under high compression ratios: Query-guided compressor for llms. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12685–12695, Bangkok, Thailand. Association for Computational Linguistics.

Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3):1–45.

Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, Bin Li, Ping Luo, Tong Lu, Yu Qiao, and Jifeng Dai. 2023. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. *arXiv preprint arXiv:2312.14238*.

Bharath Chintagunta, Namit Katariya, Xavier Amatriain, and Anitha Kannan. 2021. Medically aware gpt-3 as a data generator for medical dialogue summarization. In *Machine Learning for Healthcare Conference*, pages 354–372. PMLR.

Chengwei Dai, Kun Li, Wei Zhou, and Songlin Hu. 2024. Beyond imitation: Learning key reasoning steps from dual chain-of-thoughts in reasoning distillation.

Haixing Dai, Zhengliang Liu, Wenxiong Liao, Xiaoke Huang, Yihan Cao, Zihao Wu, Lin Zhao, Shaochen Xu, Wei Liu, Ninghao Liu, et al. 2023. Auggpt: Leveraging chatgpt for text data augmentation. *arXiv preprint arXiv:2302.13007*.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359.

Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071. PMLR.

Google Deepmind. 2024. Project astra a universal ai agent that is helpful in everyday life.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022a. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022b. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*.

P Kingma Diederik. 2014. Adam: A method for stochastic optimization. *(No Title)*.

Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2022. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5547–5569. PMLR.

Qingkai Fang, Shoutao Guo, Yan Zhou, Zhengrui Ma, Shaolei Zhang, and Yang Feng. 2024. Llama-omni: Seamless speech interaction with large language models.

Wenfeng Feng, Chuzhan Hao, Yuewei Zhang, Yu Han, and Hao Wang. 2024. Mixture-of-loras: An efficient multitask tuning for large language models. *arXiv preprint arXiv:2403.03432*.

Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*, pages 10323–10337. PMLR.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.

Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. 2023. Specializing smaller language models towards multi-step reasoning.

Vinod Ganesan, Gowtham Ramesh, and Pratyush Kumar. 2021. Supershaper: Task-agnostic super pre-training of bert models with variable hidden dimensions. *arXiv preprint arXiv:2110.04711*.

Peng Gao, Jiaming Han, Renrui Zhang, Ziyi Lin, Shijie Geng, Aojun Zhou, Wei Zhang, Pan Lu, Conghui He, Xiangyu Yue, Hongsheng Li, and Yu Qiao. 2023. Llama-adapter v2: Parameter-efficient visual instruction model. *arXiv preprint arXiv:2304.15010*.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *The Twelfth International Conference on Learning Representations*.

Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. 2020. Realtoxicityprompts: Evaluating neural toxic degeneration in language models.

Georgi Gerganov. llama.cpp.

Daniele Giunchi, Nels Numan, Elia Gatti, and Anthony Steed. 2024. DreamCodeVR: Towards Democratizing Behavior Design in Virtual Reality with Speech-Driven Programming. In *2024 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*, Orlando, USA. IEEE.

Yunhao Gou, Zhili Liu, Kai Chen, Lanqing Hong, Hang Xu, Aoxue Li, Dit-Yan Yeung, James T Kwok, and Yu Zhang. 2023. Mixture of cluster-conditional lora experts for vision-language instruction tuning. *arXiv preprint arXiv:2312.12379*.

Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. Minillm: Knowledge distillation of large language models.

Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoob, Dinesh Manocha, and Tianyi Zhou. 2024. Hallusionbench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models.

Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang, Andy Narayanan, Aonan Zhang, et al. 2024. Apple intelligence foundation language models.

Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng, Zechun Liu, Yichen Wei, and Jian Sun. 2020. Single path one-shot neural architecture search with uniform sampling. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVI 16*, pages 544–560. Springer.

Jiayi Han, Liang Du, Hongwei Du, Xiangguo Zhou, Yiwen Wu, Weibo Zheng, and Donghong Han. 2024. Slim: Let llm learn more and forget less with soft lora and identity mixture. *arXiv preprint arXiv:2410.07739*.

Alex Havrilla, Yilun Du, Chuanyang Zheng, Phillip Isola, and Joshua B. Tenenbaum. 2024. Understanding the effect of noise in llm training data with algorithmic chains of thought.

Geoffrey Hinton, Oriol Vinyals, Jeff Dean, et al. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2(7).

Namgyu Ho, Laura Schmid, and Se-Young Yun. 2023. Large language models are reasoning teachers.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes.

Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. Inference without interference: Disaggregate llm inference for mixed downstream workloads.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions.

Erik Johannes Husom, Arda Goknil, Lwin Khin Shar, and Sagar Sen. 2024. The price of prompting: Profiling energy use in large language models inference.

Ganesh Jawahar, Haichuan Yang, Yunyang Xiong, Zechun Liu, Dilin Wang, Fei Sun, Meng Li, Aasish Pappu, Barlas Oguz, Muhammad Abdul-Mageed, et al. 2023. Mixture-of-supernets: Improving weight-sharing supernet training with architecture-routed mixture-of-experts. *arXiv preprint arXiv:2306.04845*.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2019. Tinybert: Distilling bert for natural language understanding. *arXiv preprint arXiv:1909.10351*.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. Tinybert: Distilling bert for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174.

Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Steven Horng, Leo Anthony Celi, and Roger Mark. 2020. Mimic-iv. *PhysioNet. Available online at: https://physionet. org/content/mimiciv/1.0/(accessed August 23, 2021)*, pages 49–55.

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*.

Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. 2020. Unity: A general platform for intelligent agents.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR.

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2023. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*.

Rui Kong, Qiyang Li, Xinyu Fang, Qingtian Feng, Qingfeng He, Yazhu Dong, Weijun Wang, Yuanchun Li, Linghe Kong, and Yunxin Liu. 2024. Lora-switch: Boosting the efficiency of dynamic llm adapters via system-algorithm co-design. *arXiv preprint arXiv:2405.17741*.

Eldar Kurtić, Elias Frantar, and Dan Alistarh. 2024. Ziplm: Inference-aware structured pruning of language models. *Advances in Neural Information Processing Systems*, 36.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Yanis Labrak, Adrien Bazoge, Emmanuel Morin, Pierre-Antoine Gourraud, Mickael Rouvier, and Richard Dufour. 2024. Biomistral: A collection of open-source pretrained large language models for medical domains.

Hugo Laurençon, Léo Tronchon, Matthieu Cord, and Victor Sanh. 2024. What matters when building vision-language models? *arXiv preprint arXiv:2405.02246*.

Jaewook Lee, Yoel Park, and Seulki Lee. 2024a. Designing extremely memory-efficient cnns for on-device vision tasks.

Nicholas Lee, Thanakul Wattanawong, Sehoon Kim, Karttikeya Mangalam, Sheng Shen, Gopala Anumanchipali, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. 2024b. Llm2llm: Boosting llms with novel iterative data enhancement. *arXiv preprint arXiv:2403.15042*.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*.

Ming Li, Lichang Chen, Jiuhai Chen, Shwai He, Jiuxiang Gu, and Tianyi Zhou. 2024a. Selective reflection-tuning: Student-selected data recycling for llm instruction-tuning. *arXiv preprint arXiv:2402.10110*.

Ming Li, Lichang Chen, Jiuhai Chen, Shwai He, Heng Huang, Jiuxiang Gu, and Tianyi Zhou. 2023a. Reflection-tuning: Data recycling improves llm instruction-tuning. *arXiv preprint arXiv:2310.11716*.

Qinbin Li, Junyuan Hong, Chulin Xie, Jeffrey Tan, Rachel Xin, Junyi Hou, Xavier Yin, Zhun Wang, Dan Hendrycks, Zhangyang Wang, Bo Li, Bingsheng He, and Dawn Song. 2024b. Llm-pbe: Assessing data privacy in large language models.

Yanwei Li, Yuechen Zhang, Chengyao Wang, Zhisheng Zhong, Yixin Chen, Ruihang Chu, Shaoteng Liu, and Jiaya Jia. 2024c. Mini-gemini: Mining the potential of multi-modality vision language models. *arXiv preprint arXiv:2403.18814*.

Zonglin Li, Chong You, Srinadh Bhojanapalli, Daliang Li, Ankit Singh Rawat, Sashank J. Reddi, Ke Ye, Felix Chern, Felix Yu, Ruiqi Guo, and Sanjiv Kumar. 2023b. The lazy neuron phenomenon: On emergence of activation sparsity in transformers. In *The Eleventh International Conference on Learning Representations*.

Wing Lian, Bleys Goodson, Eugene Pentland, Austin Cook, Chanvichet Vong, and "Teknium". 2023. Openorca: An open dataset of gpt augmented flan reasoning traces. https://https://huggingface.co/Open-Orca/OpenOrca.

Chen Liang, Simiao Zuo, Qingru Zhang, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2023. Less is more: Task-aware layer-wise distillation for language model compression.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100.

Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. 2024a. Llava-next: Improved reasoning, ocr, and world knowledge.

Hong Liu, Zhiyuan Li, David Leo Wright Hall, Percy Liang, and Tengyu Ma. 2024b. Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *The Twelfth International Conference on Learning Representations*.

Jing Liu, Ruihao Gong, Xiuying Wei, Zhiwei Dong, Jianfei Cai, and Bohan Zhuang. 2023a. Qllm: Accurate and efficient low-bitwidth quantization for large language models. *arXiv preprint arXiv:2310.08041*.

Qidong Liu, Xian Wu, Xiangyu Zhao, Yuanshao Zhu, Derong Xu, Feng Tian, and Yefeng Zheng. 2023b. Moelora: An moe-based parameter efficient fine-tuning method for multi-task medical applications. *arXiv preprint arXiv:2310.18339*.

Shih-yang Liu, Zechun Liu, Xijie Huang, Pingcheng Dong, and Kwang-Ting Cheng. 2023c. Llm-fp4: 4-bit floating-point quantized transformers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 592–605.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2024c. Prompt injection attack against llm-integrated applications.

Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. 2023d. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*.

Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. 2024d. Spinquant–llm quantization with learned rotations. *arXiv preprint arXiv:2405.16406*.

Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, et al. 2024e. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. *arXiv:2402.14905*.

Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, et al. 2023e. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning*, pages 22137–22176. PMLR.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024f. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Gen Luo, Xue Yang, Wenhan Dou, Zhaokai Wang, Jifeng Dai, Yu Qiao, and Xizhou Zhu. 2024a. Mono-internvl: Pushing the boundaries of monolithic multimodal large language models with endogenous visual pre-training. *arXiv preprint arXiv:2410.08202*.

Tongxu Luo, Jiahe Lei, Fangyu Lei, Weihao Liu, Shizhu He, Jun Zhao, and Kang Liu. 2024b. Moelora: Contrastive learning guided mixture of experts on parameter-efficient fine-tuning for large language models. *arXiv preprint arXiv:2402.12851*.

Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, and Xiaowen Chu. Benchmarking and dissecting the nvidia hopper gpu architecture (2024). *URL https://arxiv. org/abs/2402.13499*.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720.

Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. 2023. Teaching small language models to reason.

Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? *Advances in neural information processing systems*, 32.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed precision training. In *International Conference on Learning Representations*.

Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heinz, and Dan Roth. 2021. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56:1 – 40.

Seyed Iman Mirzadeh, Keivan Alizadeh-Vahid, Sachin Mehta, Carlo C del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. ReLU strikes back: Exploiting activation sparsity in large language models. In *The Twelfth International Conference on Learning Representations*.

Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. 2024. Compact language models via pruning and knowledge distillation.

Moin Nadeem, Anna Bethke, and Siva Reddy. 2021. StereoSet: Measuring stereotypical bias in pretrained language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5356–5371, Online. Association for Computational Linguistics.

Nikita Nangia, Clara Vania, Rasika Bhalerao, and Samuel R. Bowman. 2020. CrowS-pairs: A challenge dataset for measuring social biases in masked language models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1953–1967, Online. Association for Computational Linguistics.

Deepak Narayanan, Keshav Santhanam, Peter Henderson, Rishi Bommasani, Tony Lee, and Percy Liang. 2023. Cheaply evaluating inference efficiency metrics for autoregressive transformer apis.

E. J. Nyström. 1930. Über Die Praktische Auflösung von Integralgleichungen mit Anwendungen auf Randwertaufgaben. *Acta Mathematica*, 54(none):185 – 204.

OpenAI. 2024a. Gpt-4 technical report.

OpenAI. 2024b. Hello gpt-4o.

Alicia Parrish, Angelica Chen, Nikita Nangia, Vishakh Padmakumar, Jason Phang, Jana Thompson, Phu Mon Htut, and Samuel R. Bowman. 2022. Bbq: A hand-built bias benchmark for question answering.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. 2024. Characterizing power management opportunities for llms in the cloud. In *ASPLOS '24: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, New York, NY, USA. Association for Computing Machinery.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Leon Derczynski, Xingjian Du, Matteo Grella, Kranthi Gv, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden

Lau, Jiaju Lin, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Johan Wind, Stanisław Woźniak, Zhenyuan Zhang, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. 2023. RWKV: Reinventing RNNs for the transformer era. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14048–14077, Singapore. Association for Computational Linguistics.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. *OpenAI blog*.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners.

Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. 2021. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE.

Pranav Rajpurkar, Jian Zhang, Konstantin Liu, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text.

Siva Reddy, Danqi Chen, and Christopher D Manning. 2019. Coqa: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266.

Apple Machine Learning Research. 2024. Introducing apple's on-device and server foundation models.

Anna Rogers, Olga Kovaleva, and Anna Rumshisky. 2020. A primer in BERTology: What we know about how BERT works. *Transactions of the Association for Computational Linguistics*, 8:842–866.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68.

Hassan Sajjad, Fahim Dalvi, Nadir Durrani, and Preslav Nakov. 2023. On the effect of dropping layers of pre-trained transformer models. *Computer Speech & Language*, 77:101429.

V Sanh. 2019. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Michael Santacroce, Zixin Wen, Yelong Shen, and Yuanzhi Li. 2023. What matters in the structured pruning of generative language models? *arXiv preprint arXiv:2302.03773*.

Rishov Sarkar, Hanxue Liang, Zhiwen Fan, Zhangyang Wang, and Cong Hao. 2023. Edge-moe: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts.

Paul-Edouard Sarlin, Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. 2020. Superglue: Learning feature matching with graph neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4938–4947.

Atreya Shankar, Andreas Waldis, Christof Bless, Maria Andueza Rodriguez, and Luca Mazzola. 2023. Privacyglue: A benchmark dataset for general language understanding in privacy policies. *Applied Sciences*, 13(6):3701.

Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer.

Noam Shazeer and Mitchell Stern. 2018. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pages 4596–4604. PMLR.

Tianhao Shen, Renren Jin, Yufei Huang, Chuang Liu, Weilong Dong, Zishan Guo, Xinwei Wu, Yan Liu, and Deyi Xiong. 2023. Large language model alignment: A survey. *ArXiv*, abs/2309.15025.

Xuan Shen, Peiyan Dong, Lei Lu, Zhenglun Kong, Zhengang Li, Ming Lin, Chao Wu, and Yanzhi Wang. 2024a. Agile-quant: Activation-guided quantization for faster inference of llms on the edge. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 18944–18951.

Xuan Shen, Zhenglun Kong, Changdi Yang, Zhaoyang Han, Lei Lu, Peiyan Dong, Cheng Lyu, Chih-hsiang Li, Xuehang Guo, Zhihao Shu, et al. 2024b. Edgeqat: Entropy and distribution guided quantization-aware training for the acceleration of lightweight llms on the edge. *arXiv preprint arXiv:2402.10787*.

Xuan Shen, Pu Zhao, Yifan Gong, Zhenglun Kong, Zheng Zhan, Yushu Wu, Ming Lin, Chao Wu, Xue Lin, and Yanzhi Wang. 2024c. Search for efficient large language models. *arXiv preprint arXiv:2409.17372*.

Yiping Song, Juhua Zhang, Zhiliang Tian, Yuxin Yang, Minlie Huang, and Dongsheng Li. 2024. Llm-based privacy data augmentation guided by knowledge distillation with a distribution tutor for medical text classification. *arXiv preprint arXiv:2402.16515*.

Sharath Turuvekere Sreenivas, Saurav Muralidharan, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. 2024. Llm pruning and distillation in practice: The minitron approach.

Jovan Stojkovic, Esha Choukse, Chaojie Zhang, Inigo Goiri, and Josep Torrellas. 2024a. Towards greener llms: Bringing energy-efficiency to the forefront of llm inference.

Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2024b. Dynamollm: Designing llm inference clusters for performance and energy efficiency.

Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. 2023. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*.

Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. 2020. MobileBERT: a compact task-agnostic BERT for resource-limited devices. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2158–2170, Online. Association for Computational Linguistics.

Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR.

Chaofan Tao, Lu Hou, Haoli Bai, Jiansheng Wei, Xin Jiang, Qun Liu, Ping Luo, and Ngai Wong. 2023. Structured pruning for efficient generative pre-trained language models. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10880–10895.

Jean-Loup Tastet and Inar Timiryasov. 2024. Babyllama-2: Ensemble-distilled models consistently outperform teachers with limited data. *arXiv preprint arXiv:2409.17312*.

Chameleon Team. 2024a. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*.

Gemini Team. 2024b. Gemini: A family of highly capable multimodal models.

Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Omkar Thawakar, Ashmal Vayani, Salman Khan, Hisham Cholakal, Rao M Anwer, Michael Felsberg, Tim Baldwin, Eric P Xing, and Fahad Shahbaz Khan. 2024. Mobillama: Towards accurate and lightweight fully transparent gpt. *arXiv preprint arXiv:2402.16840*.

Inar Timiryasov and Jean-Loup Tastet. 2023a. Baby llama: knowledge distillation from an ensemble of teachers trained on a small dataset with no performance penalty. *arXiv preprint arXiv:2308.02019*.

Inar Timiryasov and Jean-Loup Tastet. 2023b. Baby llama: knowledge distillation from an ensemble of teachers trained on a small dataset with no performance penalty.

Fernanda De La Torre, Cathy Mengying Fang, Han Huang, Andrzej Banburski-Fahey, Judith Amores Fernandez, and Jaron Lanier. 2024. Llmr: Real-time prompting of interactive worlds using large language models.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023b. Llama: Open and efficient foundation language models.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023c. Llama 2: Open foundation and fine-tuned chat models.

Prasetya Ajie Utama, Nafise Sadat Moosavi, and Iryna Gurevych. 2020. Towards debiasing NLU models from unknown biases. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7597–7610, Online. Association for Computational Linguistics.

Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. 2019. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the*

*57th Annual Meeting of the Association for Computational Linguistics*, pages 5797–5808, Florence, Italy. Association for Computational Linguistics.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. 2024a. Knowledge fusion of large language models.

Fanqi Wan, Longguang Zhong, Ziyi Yang, Ruijun Chen, and Xiaojun Quan. 2024b. Fusechat: Knowledge fusion of chat models.

Junyang Wang, Yuhang Wang, Guohai Xu, Jing Zhang, Yukai Gu, Haitao Jia, Jiaqi Wang, Haiyang Xu, Ming Yan, Ji Zhang, and Jitao Sang. 2024. Amber: An llm-free multi-dimensional benchmark for mllms hallucination evaluation.

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. 2020a. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*.

Ziheng Wang, Jeremy Wohlwend, and Tao Lei. 2020b. Structured pruning of large language models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6151–6162, Online. Association for Computational Linguistics.

Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. 2023. f-divergence minimization for sequence-level knowledge distillation.

Chenxi Whitehouse, Monojit Choudhury, and Alham Fikri Aji. 2023. Llm-powered data augmentation for enhanced cross-lingual performance. *arXiv preprint arXiv:2305.14288*.

Bichen Wu, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, Yangqing Jia, and Kurt Keutzer. 2019. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10734–10742.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2024. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*.

Mengzhou Xia, Zexuan Zhong, and Danqi Chen. 2022. Structured pruning learns compact and accurate models. *arXiv preprint arXiv:2204.00408*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR.

Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. 2021. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 14138–14148.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

Jin Xu, Xu Tan, Renqian Luo, Kaitao Song, Jian Li, Tao Qin, and Tie-Yan Liu. 2021. Nas-bert: task-agnostic and adaptive-size bert compression with neural architecture search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 1933–1943.

Shu Yang, Muhammad Asif Ali, Cheng-Long Wang, Lijie Hu, and Di Wang. 2024. Moral: Moe augmented lora for llms' lifelong learning. *arXiv preprint arXiv:2402.11260*.

Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems*, 35:27168–27183.

Da Yu, Peter Kairouz, Sewoong Oh, and Zheng Xu. 2024. Privacy-preserving instructions for aligning large language models.

Yuxuan Yue, Zhihang Yuan, Haojie Duanmu, Sifan Zhou, Jianlong Wu, and Liqiang Nie. 2024. Wkvquant: Quantizing weight and key/value cache for large language models gains more. *arXiv preprint arXiv:2402.12065*.

Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, et al. 2024. Flightllm: Efficient large language model inference with a complete mapping flow on fpgas. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pages 223–234.

Bohang Zhang, Jikai Jin, Cong Fang, and Liwei Wang. 2020. Improved analysis of clipping algorithms for non-convex optimization. *Advances in Neural Information Processing Systems*, 33:15511–15521.

Hongbo Zhang, Junying Chen, Feng Jiang, Fei Yu, Zhihong Chen, Jianquan Li, Guiming Chen, Xiangbo Wu, Zhiyi Zhang, Qingying Xiao, Xiang Wan, Benyou Wang, and Haizhou Li. 2023a. Huatuogpt, towards taming language models to be a doctor. *arXiv preprint arXiv:2305.15075*.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.

Renrui Zhang, Jiaming Han, Chris Liu, Peng Gao, Aojun Zhou, Xiangfei Hu, Shilin Yan, Pan Lu, Hongsheng Li, and Yu Qiao. 2023b. Llama-adapter: Efficient fine-tuning of language models with zero-init attention. *arXiv preprint arXiv:2303.16199*.

Jiaxu Zhao, Meng Fang, Shirui Pan, Wenpeng Yin, and Mykola Pechenizkiy. 2023a. Gptbias: A comprehensive framework for evaluating bias in large language models.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023b. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*.

Aojun Zhou, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, Wenxiu Sun, and Hongsheng Li. 2021. Learning n: m fine-grained structured sparse neural networks from scratch. *arXiv preprint arXiv:2102.04010*.

He Zhu, Junyou Su, Tianle Lun, Yicheng Tao, Wenjia Zhang, Zipei Fan, and Guanhua Chen. 2024. Fanno: Augmenting high-quality instruction data with open-sourced llms only. *arXiv preprint arXiv:2408.01323*.

Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2023. A survey on model compression for large language models. *ArXiv*, abs/2308.07633.

Barret Zoph and Quoc V Le. 2016. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*.

## A Further Discussion on Pruning Techniques

For unstructured pruning for SLMs, we further note that Wanda (Sun et al., 2023) incorporates both weights and activations into consideration during pruning process, and eliminates the need of weight updates. In addition, the n:m pruning strategy (Zhou et al., 2021) brings unstructured pruning to model acceleration by pruning exactly $n$ weights out of every $m$, balancing pruning flexibility and computational efficiency for significant speedups. NVIDIA's TensorRT leverages such sparse patterns to optimize memory access and reduce computational loads, accelerating inference on GPUs, particularly hardware like the A100. Additionally, the n:m sparse pattern can also be applied in edge AI applications on NVIDIA Jetson Nano to enhance power efficiency and optimize model size. Finally, unstructured pruning often results in sparse matrices requiring specialized hardware or algorithms to maximize computational benefits (Frantar and Alistarh, 2023).